

Ticket Resilience Lab

Mecanismos de tolerancia a fallas en Kubernetes

Integrantes: Alexander y Jean Pierre

Repositorio: <https://github.com/jean-pierre-valarezo/ticket-resilience-lab.git>

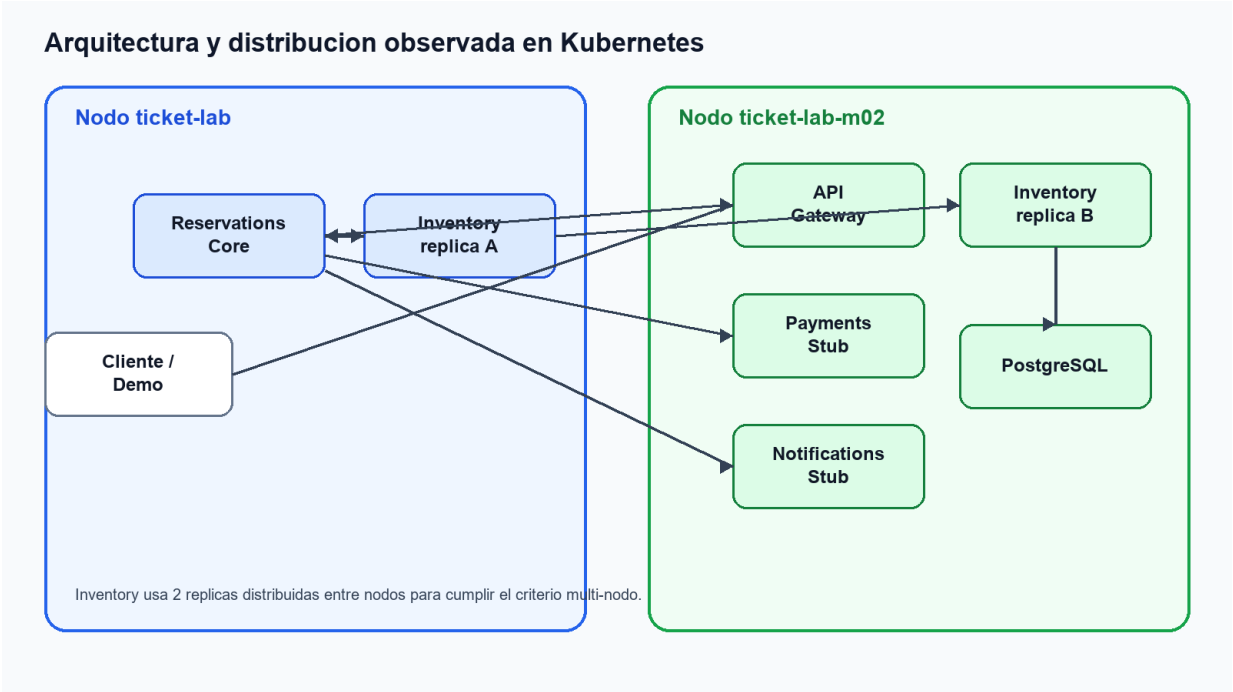
Fecha: 2026-07-14

1. Objetivo

Implementar y demostrar mecanismos de tolerancia a fallos sobre una arquitectura simplificada de venta de entradas desplegada en Kubernetes. La practica provoca fallos reales sobre un cluster de dos nodos y verifica, con evidencia propia, que el sistema responde de forma controlada o se recupera sin colapsar.

2. Arquitectura

La solucion usa microservicios comunicados por REST: gateway, reservations, inventory, payments, notifications y postgres.



Inventory se despliega con 2 replicas y topologySpreadConstraints por hostname para distribuirlo entre nodos.

3. Despliegue Kubernetes

Elemento	Valor
Cluster	minikube perfil ticket-lab
Nodos	ticket-lab y ticket-lab-m02
Namespace	ticket-lab
Componente critico	inventory con 2 replicas
Manifiestos	k8s/base

4. Mapeo de fallos

Fallo	Tipo	Inyeccion	Defensa	Estado
Inventario Fantasma	Disponibilidad	Escalar deployment/inventory a 0 replicas	Retry con backoff y error controlado	Implementado
Pasarela Lenta	Latencia	PAYMENT_DELAY_MS=20000	Timeout + circuit breaker	Implementado
Diluvio de Peticiones	Sobrecarga	25 solicitudes consecutivas al gateway	Rate limiting	Implementado
Base de Datos Intermitente	Conectividad	Flapping/reinicio controlado de PostgreSQL	Exponential backoff, idempotencia	Analisis teorico
Correo Perdido	Fallo no critico	Escalar deployment/notifications a 0 replicas	Fallback pending	Implementado
Condicion de Carrera	Consistencia	Compras concurrentes sobre un mismo item	Transacciones y bloqueo pesimista	Analisis teorico

5. Mecanismos implementados

Inventario Fantasma

Falla inyectada: El servicio inventory se escala a 0 replicas.

Defensa: reservations aplica retry con backoff y responde con un error controlado si no puede validar inventario.

Resultado: La reserva falla con HTTP 503 y no se confirma una reserva sin inventario.

Evidencia: 09-chaos-inventory-down-http-503.png

Pasarela Lenta

Falla inyectada: payments simula 20 segundos de latencia con PAYMENT_DELAY_MS=20000.

Defensa: reservations usa timeout y circuit breaker. Despues de tres fallos, el circuito pasa a open.

Resultado: Los intentos devuelven HTTP 503 y /resilience muestra payment_circuit.state = open.

Evidencia: 11-chaos-payments-slow-circuit-breaker-open-1.png, 12-chaos-payments-slow-circuit-breaker-open-2.png

Correo Perdido

Falla inyectada: notifications se escala a 0 replicas.

Defensa: La notificacion se trata como dependencia no critica. La reserva se conserva y la notificacion queda pending.

Resultado: La respuesta es HTTP 200, status confirmed y notification.status pending.

Evidencia: 15-chaos-notifications-down-fallback-pending.png

Diluvio de Peticiones

Falla inyectada: Se envian 25 solicitudes consecutivas contra POST /reserve.

Defensa: gateway aplica rate limiting por cliente dentro de una ventana temporal.

Resultado: El sistema responde HTTP 429 cuando se supera el limite de peticiones.

Evidencia: 17-chaos-traffic-spike-rate-limit-429.png

6. Analisis teorico de fallos no implementados

Base de Datos Intermitente

Una base de datos intermitente afecta operaciones de escritura y obliga a decidir entre disponibilidad y consistencia durante una partición. En reservas, confirmar sin escritura durable puede producir ventas fantasma.

Solución propuesta: base de datos de alta disponibilidad, failover, pool de conexiones, retries con backoff y jitter, idempotency keys y patron outbox.

Condición de Carrera

La condición de carrera aparece cuando dos clientes reservan el mismo asiento simultáneamente. Sin aislamiento transaccional, ambos pueden observarlo como disponible.

Solución propuesta: transacciones ACID, SELECT FOR UPDATE, restricción única por asiento activo y holds temporales con expiración.

7. Conclusiones

El laboratorio evidencia que cada patrón debe ubicarse en el punto correcto: retry con backoff para dependencias transitorias, circuit breaker para servicios lentos, fallback para dependencias no críticas y rate limiting en el borde del sistema.

Anexo: evidencias clave

```
PS C:\Users\Pc\Documents\ticket-resilience-lab> kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
KERNEL-VERSION   CONTAINER-RUNTIME
ticket-lab        Ready    control-plane   3h48m   v1.35.1   192.168.58.2   <none>        Debian GNU/Linux 12 (bookworm)
6.6.87.2-microsoft-standard-WSL2   docker://29.2.1
ticket-lab-m02    Ready    <none>         3h47m   v1.35.1   192.168.58.3   <none>        Debian GNU/Linux 12 (bookworm)
6.6.87.2-microsoft-standard-WSL2   docker://29.2.1
PS C:\Users\Pc\Documents\ticket-resilience-lab> kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
KERNEL-VERSION   CONTAINER-RUNTIME
ticket-lab        Ready    control-plane   4h2m   v1.35.1   192.168.58.2   <none>        Debian GNU/Linux 12 (bookworm)
6.6.87.2-microsoft-standard-WSL2   docker://29.2.1
ticket-lab-m02    Ready    <none>         4h1m   v1.35.1   192.168.58.3   <none>        Debian GNU/Linux 12 (bookworm)
6.6.87.2-microsoft-standard-WSL2   docker://29.2.1
PS C:\Users\Pc\Documents\ticket-resilience-lab> |
```

Cluster Kubernetes con dos nodos Ready

```
PS C:\Users\Pc\Documents\ticket-resilience-lab> kubectl -n ticket-lab get pods -o wide
NAME                                READY    STATUS    RESTARTS   AGE    IP             NODE             NOMINATED NODE    REA
DINESS GATES
gateway-6c849dccc6-48nm2            1/1     Running   0           4h31m   10.244.1.2     ticket-lab-m02   <none>             <no
ne>
inventory-7d6f5868b7-hh7ss          1/1     Running   0           4m15s   10.244.0.7     ticket-lab        <none>             <no
ne>
inventory-7d6f5868b7-v8j89          1/1     Running   0           4m27s   10.244.1.17    ticket-lab-m02   <none>             <no
ne>
notifications-7699c97d7-d2bt2       1/1     Running   0           18m     10.244.1.16    ticket-lab-m02   <none>             <no
ne>
payments-559ff975d5-zvndx           1/1     Running   0           20m     10.244.1.14    ticket-lab-m02   <none>             <no
ne>
postgres-754697f674-68w96          1/1     Running   0           4h31m   10.244.1.7     ticket-lab-m02   <none>             <no
ne>
reservations-db959d9c4-shq5r        1/1     Running   0           20m     10.244.1.15    ticket-lab-m02   <none>             <no
ne>
```

Pods desplegados con inventory replicado entre nodos

```
{
  {
    "seat_id": "A8",
    "status": "available",
    "reserved_by": null
  },
  {
    "seat_id": "A9",
    "status": "available",
    "reserved_by": null
  }
}

=== Reserva exitosa via gateway ===
HTTP 200
{
  "reservation_id": "2386d5a0-8546-450b-8d25-83ef06a1a961",
  "status": "confirmed",
  "inventory": {
    "event_id": "concert-1",
    "seat_id": "A2",
    "status": "reserved",
    "reserved_by": "demo-user"
  },
  "payment": {
    "status": "charged",
    "transaction_id": "d46b1582-aa97-424f-b541-caa87db87625",
    "reservation_id": "2386d5a0-8546-450b-8d25-83ef06a1a961",
    "amount": 50.0
  },
}
```

Reserva base confirmada

```
PS C:\Users\Pc\Documents\ticket-resilience-lab> .\scripts\05-chaos-inventory-down.ps1
Inyectando falla: Inventory down.
deployment.apps/inventory scaled
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE REA
DINESS GATES
gateway-6c849dccc6-48nm2 1/1 Running 0 4h6m 10.244.1.2 ticket-lab-m02 <none> <no
ne>
notifications-7699c97d7-z2x6t 1/1 Running 0 3h59m 10.244.1.11 ticket-lab-m02 <none> <no
ne>
payments-559ff975d5-59lqz 1/1 Running 0 4h 10.244.1.9 ticket-lab-m02 <none> <no
ne>
postgres-754697f674-68w96 1/1 Running 0 4h6m 10.244.1.7 ticket-lab-m02 <none> <no
ne>
reservations-6c89b446cc-ss5dn 1/1 Running 0 3h58m 10.244.0.5 ticket-lab <none> <no
ne>

=== Reserva con Inventory caido ===
HTTP 503
```

Inventario caido con HTTP 503

```
PS C:\Users\Pc\Documents\ticket-resilience-lab> .\scripts\07-chaos-payments-slow.ps1
Inyectando falla: pasarela de pagos lenta con 20 segundos de latencia.
deployment.apps/payments env updated
Waiting for deployment "payments" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "payments" rollout to finish: 1 old replicas are pending termination...
deployment "payments" successfully rolled out

=== Intento 1 con pagos lentos ===
HTTP 503

=== Intento 2 con pagos lentos ===
HTTP 503

=== Intento 3 con pagos lentos ===
HTTP 503

=== Intento 4 con pagos lentos ===
HTTP 503

=== Estado del circuit breaker de pagos ===
HTTP 200
{
  "payment_circuit": {
    "state": "open",
    "failure_count": 3
  }
}
```

Pagos lentos con circuit breaker abierto

```
PS C:\Users\Pc\Documents\ticket-resilience-lab> .\scripts\09-chaos-notifications-down.ps1
Inyectando falla: Notifications down.
deployment.apps/notifications scaled
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
gateway-8655c8f889-lch5 1/1 Running 0 40s 10.244.1.29 ticket-lab-m02 <none> <none>
inventory-5544cd6f6-cjq8f 1/1 Running 0 27s 10.244.1.30 ticket-lab-m02 <none> <none>
inventory-5544cd6f6-sllcf 1/1 Running 0 40s 10.244.0.16 ticket-lab <none> <none>
payments-5d6ff89484-4zwv7 1/1 Running 0 40s 10.244.1.28 ticket-lab-m02 <none> <none>
postgres-7d4fd86d89-xfcq6 1/1 Running 0 40s 10.244.1.27 ticket-lab-m02 <none> <none>
reservations-6874999577-k7tdn 1/1 Running 0 40s 10.244.0.17 ticket-lab <none> <none>

=== Reserva con Notifications caido ===
HTTP 200
{
  "reservation_id": "a0a213c5-8434-440c-a48c-bfadfa38fddd",
  "status": "confirmed",
  "inventory": {
    "event_id": "concert-1",
    "seat_id": "A7",
    "status": "reserved",
    "reserved_by": "notification-down-demo"
  },
  "payment": {
    "status": "charged",
    "transaction_id": "3beff8c8-a2aa-4c0d-97f6-3b68be2fcf04",
    "reservation_id": "a0a213c5-8434-440c-a48c-bfadfa38fddd",
    "amount": 50.0
  },
  "notification": {
    "status": "pending",
    "reason": "notification service unavailable"
  }
}

Resultado esperado: reserva confirmada y notification.status=pending.
```

Notificaciones caidas con fallback pending

```
PS C:\Users\Pc\Documents\ticket-resilience-lab> .\scripts\11-chaos-traffic-spike.ps1
Inyectando falla: diluvio de peticiones contra el gateway.
Se esperan respuestas 429 cuando el rate limiter corta el exceso de trafico.
Intento 1: HTTP 200
Intento 2: HTTP 409
Intento 3: HTTP 409
Intento 4: HTTP 409
Intento 5: HTTP 409
Intento 6: HTTP 409
Intento 7: HTTP 409
Intento 8: HTTP 409
Intento 9: HTTP 409
Intento 10: HTTP 409
Intento 11: HTTP 409
Intento 12: HTTP 409
Intento 13: HTTP 429
Intento 14: HTTP 429
Intento 15: HTTP 429
Intento 16: HTTP 429
Intento 17: HTTP 429
Intento 18: HTTP 429
Intento 19: HTTP 429
Intento 20: HTTP 429
Intento 21: HTTP 429
Intento 22: HTTP 429
Intento 23: HTTP 429
Intento 24: HTTP 429
Intento 25: HTTP 429

Resumen por codigo HTTP:

Name Count
----
200      1
409     11
429     13
```

Diluvio de peticiones con HTTP 429